



Môn Học: NHẬP MÔN TRÍ TUỆ NHÂN TẠO

**Đề Tài: Nghiên Cứu Và Xây Dựng Hệ Thống
Trí Tuệ Nhân Tạo Cho Trò Chơi Cờ Vua**

Giảng Viên :

PGS.TS Nguyễn Thế Toàn
Thầy Vi Anh Quân

Nhóm : 4

Tên Thành Viên :

Nguyễn Thị Hà	23001602
Phạm Minh Trí	23001634
Nguyễn Tuấn Đạt	23001594
Trần Danh Đoàn	23001597

PHÂN CHIA CÔNG VIỆC

Thành viên	Vai trò	Đóng góp
Nguyễn Thị Hà	<u>Viết code</u>	25%
Phạm Minh Trí	<u>Hỗ trợ kiểm thử code và</u> <u>thuyết trình</u>	25%
Nguyễn Tuấn Đạt	Tổng hợp nội dung, Làm báo cáo	25%
Trần Danh Đoàn	Tổng hợp nội dung, làm slide	25%

I. GIỚI THIỆU VỀ ĐỀ TÀI

II. CƠ SỞ LÝ THUYẾT

III. XÂY DỰNG CHƯƠNG TRÌNH

IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

V. HƯỚNG PHÁT TRIỂN

I. GIỚI THIỆU VỀ ĐỀ TÀI

1. Mục tiêu đề tài

- Xây dựng chương trình cờ vua có khả năng tự động đưa ra nước đi bằng trí tuệ nhân tạo (AI).
- Mô phỏng quá trình suy nghĩ và ra quyết định của người chơi cờ.
- Nghiên cứu và ứng dụng các thuật toán tìm kiếm trong AI

2. Ý nghĩa đề tài

- Minh họa ứng dụng của AI trong trò chơi chiến lược
- Giúp hiểu rõ cơ chế ra quyết định của máy tính
- Là nền tảng cho các hệ thống AI phức tạp hơn

3. Chức năng chính

- Hiển thị bàn cờ và các quân cờ
- Cho phép người chơi đấu với máy
- AI tự động phân tích và lựa chọn nước đi tối ưu
- Đánh giá trạng thái bàn cờ theo thời gian thực



II. CƠ SỞ LÝ THUYẾT

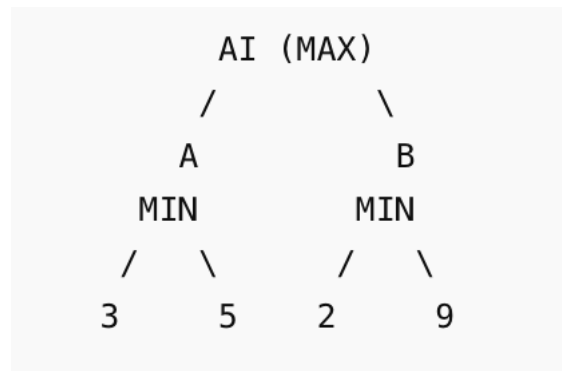
1. Thuật toán Minimax và Cây Trò Chơi(Game Tree)

1.1. Khái niệm:

- **Minimax** là thuật toán giúp AI lựa chọn nước đi tối ưu bằng cách giả định rằng đối thủ cũng luôn chơi nước đi tốt nhất. Thuật toán sẽ sinh ra tất cả các nước đi hợp lệ từ trạng thái bàn cờ hiện tại. Sau đó, với mỗi nước đi này, AI tiếp tục giả lập các phản ứng có thể của đối thủ. Quá trình này được lặp lại nhiều lần, tạo thành một **cây trò chơi (Game Tree)**, trong đó mỗi nút biểu diễn một trạng thái bàn cờ và mỗi nhánh biểu diễn một nước đi.

1.2. Nguyên lý hoạt động

- Giả sử AI có hai lựa chọn là nước đi A và nước đi B. Sau khi mô phỏng các phản ứng của đối thủ và đánh giá các trạng thái tương lai, thuật toán thu được kết quả như hình bên:

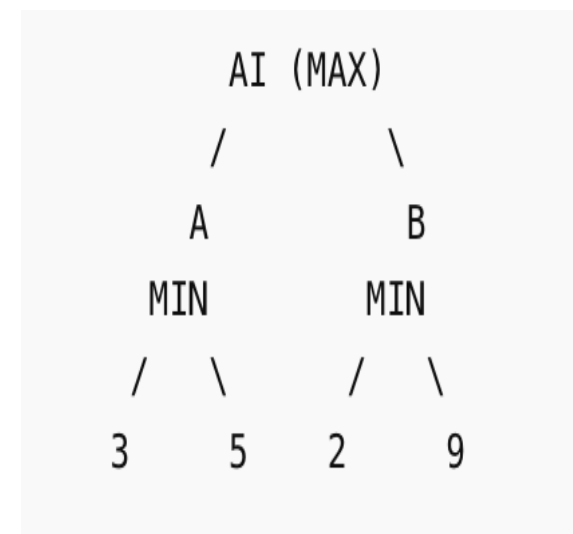


II. CƠ SỞ LÝ THUYẾT

1. Thuật toán Minimax và Cây Trò Chơi(*Game Tree*)

1.2. Nguyên lý hoạt động

- Đối với nhánh A, đối thủ sẽ chọn giá trị nhỏ nhất là **3** thay vì 5 để gây bất lợi cho AI.
- Đối với nhánh B, đối thủ sẽ chọn giá trị nhỏ nhất là **2** thay vì **9**.
- Lúc này AI so sánh hai kết quả nhận được là **3** và **2**. Vì AI đóng vai trò MAX nên sẽ chọn giá trị lớn hơn là **3**, tương ứng với nước đi **A**.



Qua ví dụ này có thể thấy Minimax luôn giả định rằng đối thủ chơi tối ưu, từ đó lựa chọn nước đi an toàn và hiệu quả nhất cho AI. Đây chính là nguyên lý cốt lõi giúp thuật toán được ứng dụng rộng rãi trong các chương trình cờ vua.

II. CƠ SỞ LÝ THUYẾT

1. Thuật toán Minimax và Cây Trò Chơi(*Game Tree*)

1.3. Đánh giá thuật toán

- Dễ triển khai và dễ hiểu, đảm bảo tìm được nước đi tối ưu trong phạm vi tìm kiếm
- Tuy nhiên, số lượng trạng thái tăng rất nhanh theo độ sâu tìm kiếm. Tốn nhiều thời gian và tài nguyên tính toán. Vì vậy, cần kết hợp với thuật toán Alpha-Beta Pruning để tăng tốc.

II. CƠ SỞ LÝ THUYẾT

2. Thuật toán *Alpha – Beta Pruning*

2.1. Khái niệm :

- Alpha–Beta Pruning là một kỹ thuật tối ưu hóa của thuật toán Minimax, được sử dụng để giảm số lượng trạng thái cần đánh giá trong cây trò chơi. Thuật toán này giúp AI cờ vua tìm kiếm nhanh hơn nhưng vẫn đảm bảo đưa ra cùng một kết quả như Minimax thông thường.

- Trong thực tế, cây trạng thái của cờ vua có số lượng nhánh rất lớn. Nếu thuật toán Minimax phải đánh giá toàn bộ các trạng thái có thể xảy ra thì thời gian xử lý sẽ rất cao. Alpha–Beta Pruning giải quyết vấn đề này bằng cách loại bỏ những nhánh chắc chắn không ảnh hưởng đến quyết định cuối cùng của AI.

II. CƠ SỞ LÝ THUYẾT

2. Thuật toán Alpha – Beta Pruning

2.2. Nguyên lý hoạt động :

Alpha–Beta Pruning hoạt động dựa trên hai giá trị:

- **Alpha (α)**: Giá trị tốt nhất mà người chơi MAX (AI) có thể đạt được tại thời điểm hiện tại.
- **Beta (β)**: Giá trị tốt nhất mà người chơi MIN (đối thủ) có thể đạt được tại thời điểm hiện tại.
- Trong quá trình duyệt cây trò chơi, thuật toán liên tục cập nhật hai giá trị α và β . Nếu tại một nút nào đó xuất hiện điều kiện:

$$\alpha \geq \beta$$

thì các nhánh còn lại của nút đó sẽ không cần được xét tiếp vì chúng không thể làm thay đổi quyết định cuối cùng. Quá trình loại bỏ các nhánh không cần thiết này được gọi là **Pruning (cắt tỉa)**.

- Nhờ đó, AI có thể tập trung vào những nước đi tiềm năng nhất thay vì phải đánh giá toàn bộ cây trạng thái.

II. CƠ SỞ LÝ THUYẾT

2. Thuật toán Alpha – Beta Pruning

2.3. Ví dụ minh họa :

- Thuật toán sẽ xét nhánh bên trái trước:

MIN chọn giá trị nhỏ nhất giữa 3 và 5 là **3**.

Giá trị Alpha tại nút MAX được cập nhật thành **3**.

- Tiếp theo xét nhánh MIN bên phải:

Giá trị đầu tiên thu được là **2**.

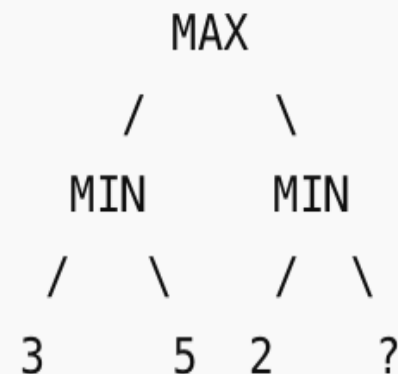
Beta tại nút MIN được cập nhật thành **2**.

- Lúc này:

$$\alpha=3 \quad \beta=2$$

- Do: $\alpha > \beta$

nên thuật toán biết rằng dù giá trị còn lại của nhánh bên phải là bao nhiêu thì MAX vẫn sẽ chọn nhánh có giá trị 3. Vì vậy nhánh còn lại không cần đánh giá nữa và được cắt bỏ.



II. CƠ SỞ LÝ THUYẾT

2. *Thuật toán Alpha – Beta Pruning*

2.4. Ứng dụng trong dự án AI cờ vua :

Trong hệ thống AI cờ vua, thuật toán Alpha–Beta Pruning được kết hợp với Minimax để tối ưu quá trình tìm kiếm. Minimax chịu trách nhiệm đánh giá và lựa chọn nước đi tối ưu, trong khi Alpha–Beta Pruning loại bỏ các nhánh không cần thiết của cây trò chơi. Sự kết hợp này giúp giảm thời gian tính toán, tăng tốc độ phản hồi và cho phép AI phân tích sâu hơn các tình huống trên bàn cờ, từ đó nâng cao chất lượng và độ chính xác của các nước đi được lựa chọn.

III. XÂY DỰNG CHƯƠNG TRÌNH

1. Xây dựng bàn cờ (Board Representation)

- Hệ thống sử dụng thư viện python-chess để biểu diễn và quản lý trạng thái bàn cờ vua. Bàn cờ gồm 64 ô cờ theo chuẩn quốc tế. Mỗi ô có thể chứa một quân cờ hoặc để trống.

- Các loại quân cờ được sử dụng trong hệ thống bao gồm:

Pawn (Tốt)

Knight (Mã)

Bishop (Tượng)

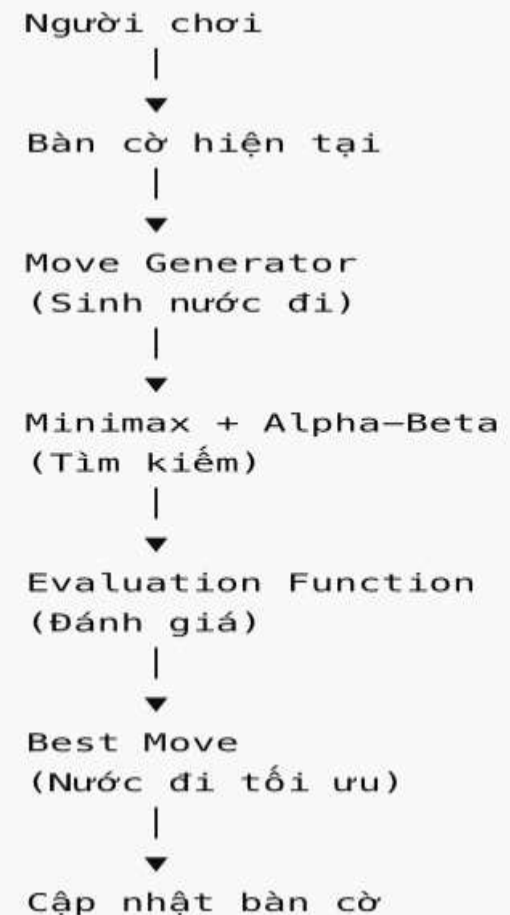
Rook (Xe)

Queen (Hậu)

King (Vua)

- Thư viện python-chess cung cấp các hàm hỗ trợ truy xuất vị trí quân cờ, sinh nước đi hợp lệ, kiểm tra chiếu, chiếu hết, hòa cờ và cập nhật trạng thái ván đấu.

- Sau mỗi nước đi, trạng thái bàn cờ được cập nhật tự động để phục vụ quá trình tìm kiếm nước đi tối ưu và đánh giá vị trí bằng thuật toán Minimax kết hợp Alpha-Beta Pruning.

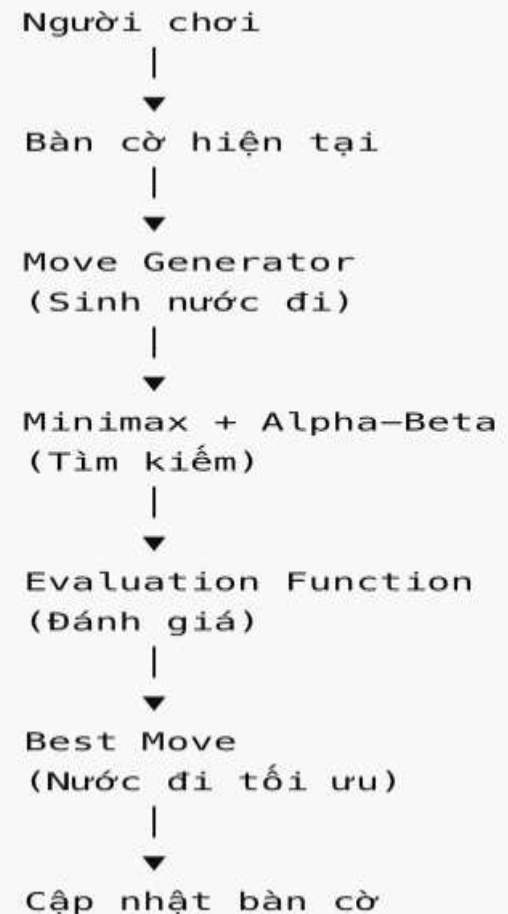


Kiến trúc chương trình

III. XÂY DỰNG CHƯƠNG TRÌNH

2. Sinh nước đi (Move Generation)

- Hệ thống sử dụng thư viện ****python-chess**** để sinh ra tất cả các nước đi hợp lệ từ trạng thái bàn cờ hiện tại.
- Bộ sinh nước đi của thư viện tự động kiểm tra và đảm bảo tuân thủ đầy đủ luật cờ vua, bao gồm:
 - +Di chuyển quân đúng quy tắc
 - +Không để vua ở trạng thái bị chiếu sau khi thực hiện nước đi
 - +Bắt quân
 - +Nhập thành
 - +Phong cấp tốt
 - +Bắt tốt qua đường (En Passant)
- Tập các nước đi hợp lệ được tạo ra sẽ là đầu vào cho thuật toán Minimax kết hợp Alpha-Beta Pruning để tìm nước đi tối ưu
- Việc sử dụng python-chess giúp đảm bảo tính chính xác của luật cờ vua và giảm độ phức tạp khi xây dựng hệ thống AI.

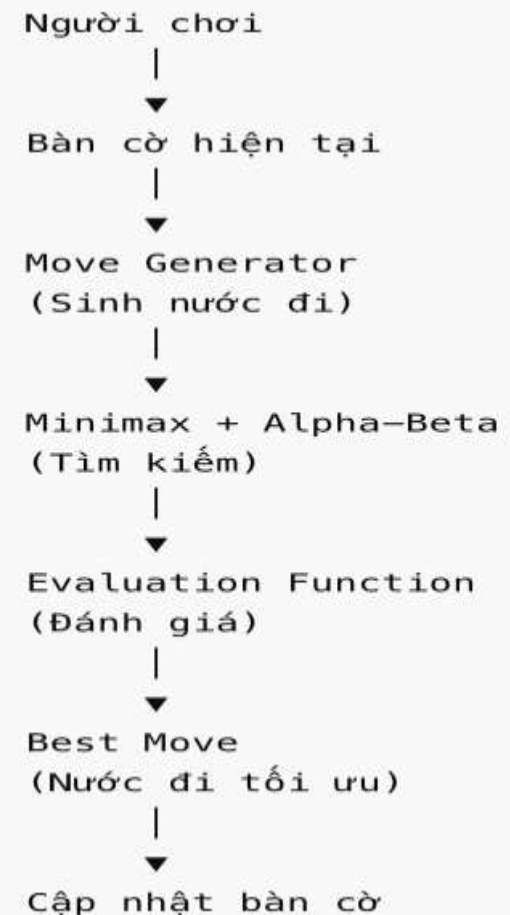


Kiến trúc chương trình

III. XÂY DỰNG CHƯƠNG TRÌNH

3. Thuật toán tìm kiếm

- Sau khi có danh sách nước đi hợp lệ, AI sử dụng thuật toán Minimax để dự đoán các diễn biến trong tương lai.
- Trong đó:
 - AI đóng vai trò MAX.
 - Đối thủ đóng vai trò MIN.
- Để tăng hiệu suất, hệ thống kết hợp:
 - Alpha-Beta Pruning.
 - Move Ordering.
 - Quiescence Search.
 - Check Extension.
- Nhờ đó AI có thể tìm kiếm sâu hơn và đưa ra quyết định tốt hơn trong cùng khoảng thời gian xử lý.



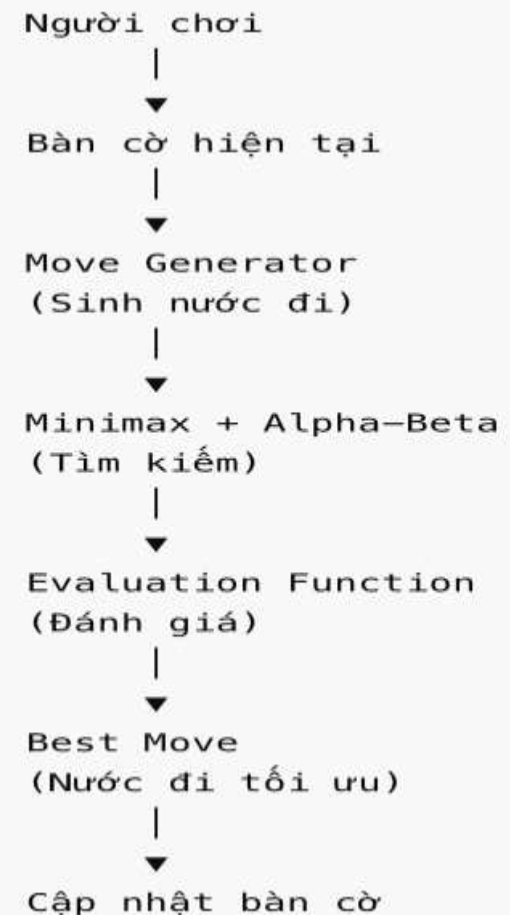
Kiến trúc chương trình

III. XÂY DỰNG CHƯƠNG TRÌNH

4. Hàm đánh giá(*Evaluation Function*)

- **Evaluation Function** là thành phần quan trọng nhất của AI cờ vua, có nhiệm vụ đánh giá mức độ có lợi hoặc bất lợi của một trạng thái bàn cờ. Giá trị trả về của hàm Evaluation là một điểm số. Điểm số càng lớn thì vị trí càng có lợi cho AI, ngược lại điểm số càng nhỏ thì vị trí càng bất lợi.
- Giá trị bằng tổng của :
 - + Material (Giá trị quân cờ)
 - + Positional Score (Điểm vị trí)
 - + Mobility (Khả năng cơ động)
 - + King Safety (An toàn của vua)
 - + Pawn Structure (Cấu trúc tốt)

Sự kết hợp các yếu tố trên giúp AI đánh giá tương đối chính xác ưu thế của từng bên trên bàn cờ, từ đó hỗ trợ thuật toán Minimax lựa chọn nước đi phù hợp.

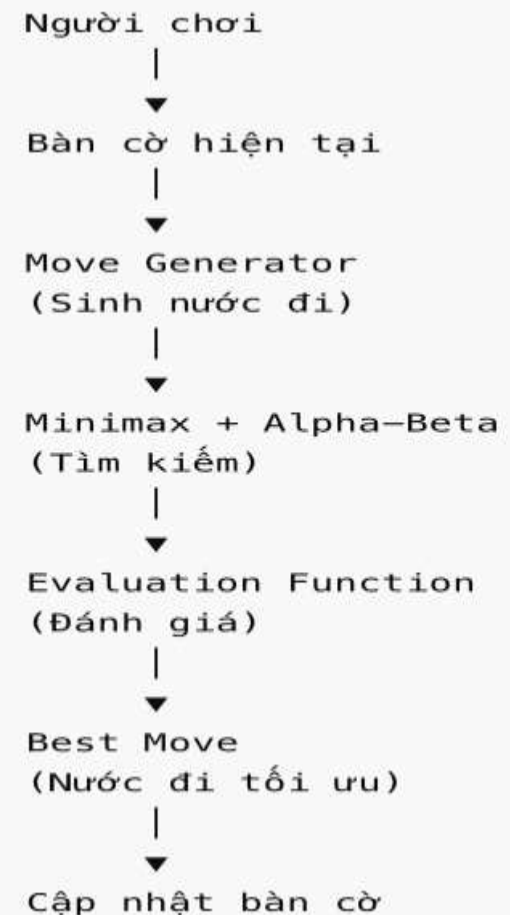


Kiến trúc chương trình

III. XÂY DỰNG CHƯƠNG TRÌNH

5. Lựa chọn nước đi tối ưu

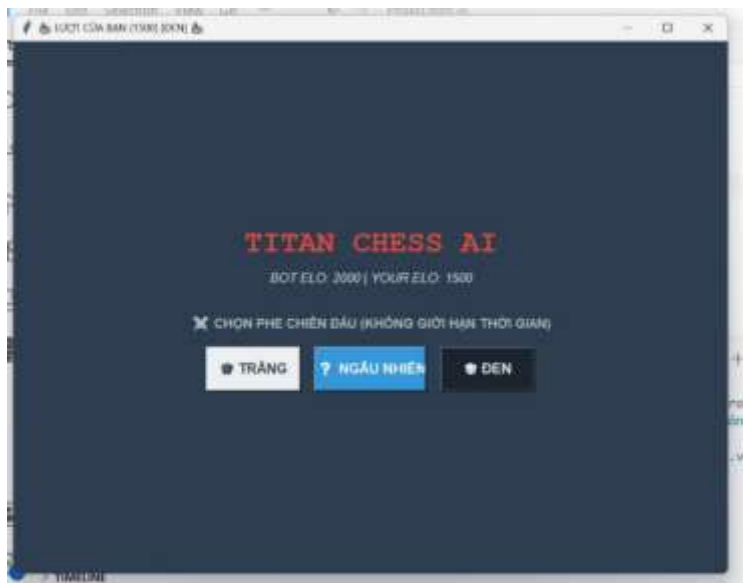
- Sau khi hoàn tất quá trình tìm kiếm và đánh giá, AI sẽ so sánh điểm số của các phương án.
- Nước đi có giá trị đánh giá cao nhất sẽ được lựa chọn và thực hiện trên bàn cờ.
- Quá trình này được lặp lại sau mỗi lượt chơi cho đến khi ván đấu kết thúc.



Kiến trúc chương trình

IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

1. Một số hình ảnh thực nghiệm



Màn hình chọn phe



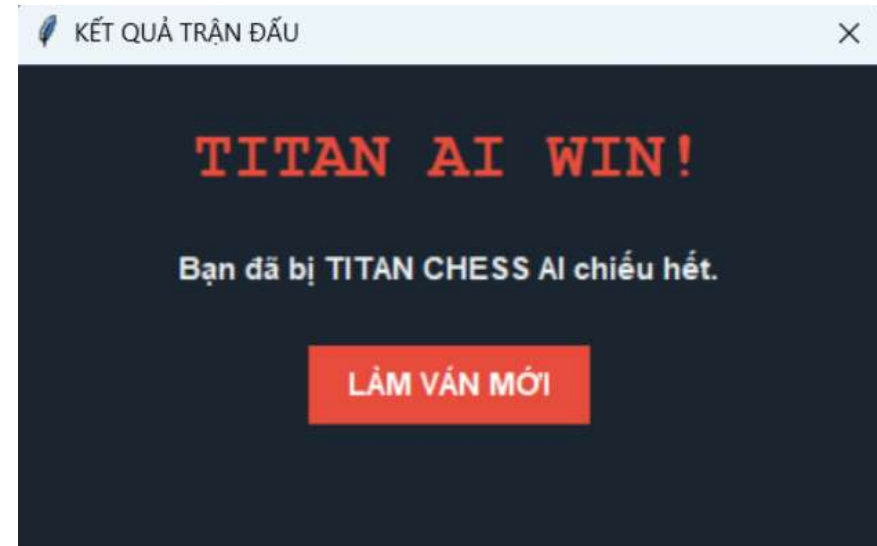
Màn hình chơi chính

IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

1. Một số hình ảnh thực nghiệm



Phân tích ván đấu



Kết quả ván đấu

IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

2. Đánh giá

2.1. Kết quả

Hệ thống được kiểm thử thông qua nhiều ván đấu với người chơi ở các mức độ khác nhau nhằm đánh giá khả năng hoạt động ổn định của giao diện, tính đúng đắn của thuật toán tìm kiếm và chất lượng nước đi được lựa chọn.

Quá trình kiểm thử tập trung vào khả năng xử lý các tình huống đặc biệt trong cờ vua, hiệu năng tìm kiếm nước đi và độ ổn định của chương trình trong quá trình sử dụng.

Dựa trên các ván đấu thử nghiệm và đánh giá bằng công cụ phân tích của Chess.com, sức mạnh của AI được ước lượng tương đương người chơi nghiệp dư đến trung cấp, khoảng 1400–1700 Elo Chess.com. Đây chỉ là ước lượng tham khảo, không phải kết quả đo Elo chính thức.

Bảng các mục kiểm thử :

Mục kiểm thử	Kết quả	Ghi chú
Tìm kiếm nước đi bằng Minimax + Alpha-Beta	Đạt	AI lựa chọn được nước đi hợp lệ trong mọi trạng thái kiểm thử
Phát hiện chiếu hết	Đạt	Xử lý chính xác các tình huống kết thúc ván cờ
Phát hiện hòa cờ	Đạt	Hỗ trợ các trạng thái hòa được thư viện python-chess cung cấp
Hiệu năng tìm kiếm (depth = 4)	2–5 giây	Phụ thuộc số lượng nước đi khả dụng
Đa luồng giao diện	Đạt	Giao diện không bị treo khi AI đang tính toán
Lưu ván cờ PGN	Đạt	Xuất được file PGN để lưu và phân tích lại
Hoàn tác nước đi	Đạt	Khôi phục chính xác trạng thái bàn cờ trước đó
Đầu hàng	Đạt	Cập nhật kết quả ván đấu chính xác

IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

2. Đánh giá

2.2. Hạn chế

- AI sử dụng hàm đánh giá heuristic được thiết kế thủ công nên chưa thể đánh giá chính xác mọi vị trí phức tạp như các chess engine hiện đại.
- Độ sâu tìm kiếm còn hạn chế (depth = 4) do giới hạn thời gian tính toán và tài nguyên phần cứng, vì vậy AI có thể bỏ sót các kế hoạch chiến thuật dài.
- Hệ thống chưa sử dụng cơ sở dữ liệu khai cuộc (Opening Book), do đó trong giai đoạn khai cuộc AI chưa đạt hiệu quả tối ưu.
- Hệ thống chưa tích hợp các kỹ thuật nâng cao như Transposition Table, Iterative Deepening hoặc Machine Learning để tăng tốc độ và chất lượng tìm kiếm.
- Chất lượng nước đi phụ thuộc nhiều vào hàm đánh giá hiện tại, do đó trong một số tình huống AI vẫn có thể đưa ra những nước đi chưa tối ưu.
- Chương trình được xây dựng với mục tiêu phục vụ học tập và nghiên cứu thuật toán AI cơ bản, chưa hướng tới khả năng cạnh tranh với các chess engine chuyên nghiệp như Stockfish hoặc Leela Chess Zero.

V. HƯỚNG PHÁT TRIỂN

1. Nâng cao hàm đánh giá (Evaluation Function)

- Hiện tại, việc đánh giá bàn cờ chủ yếu dựa trên giá trị quân cờ, vị trí quân và một số yếu tố chiến thuật cơ bản. Trong tương lai, hàm Evaluation có thể được cải tiến bằng cách bổ sung thêm các tiêu chí như:

Khả năng kiểm soát trung tâm bàn cờ.

Chất lượng cấu trúc tốt.

Mức độ an toàn của vua.

Khả năng phối hợp giữa các quân cờ.

Đánh giá các thế tấn công và phòng thủ phức tạp.

- Việc cải thiện hàm đánh giá sẽ giúp AI đưa ra những quyết định gần với tư duy của người chơi chuyên nghiệp hơn.

2. Ứng dụng Machine Learning và Deep Learning

- Hiện nay AI chủ yếu hoạt động dựa trên các thuật toán tìm kiếm truyền thống. Trong tương lai có thể kết hợp với các mô hình học máy để nâng cao khả năng đánh giá bàn cờ.

- Một số hướng phát triển:

Huấn luyện mạng Neural Network để thay thế Evaluation Function.

Học từ dữ liệu các ván đấu chuyên nghiệp.

Áp dụng Reinforcement Learning (Học tăng cường).

Xây dựng hệ thống tương tự AlphaZero.

- Cách tiếp cận này giúp AI tự học chiến thuật mới thay vì chỉ dựa vào các quy tắc được lập trình sẵn.

Thank you!